ShopMates

AI-ASSISTED DEVELOPMENT / PROJECT DESCRIPTION

A shared list.
A simpler shop.

A self-hosted family shopping list that keeps household members in sync, from planning the basket to checking out.

PROJECT TYPE	STACK	AI ASSISTANT
Full-stack web application	**React · TypeScript · Express · PostgreSQL**	**GitHub Copilot in VS Code**

01 Idea & system requirements

"ShopMates" replaces scattered notes and messages with one private, shared list for a household.

The problem

Family members often shop at different times and need the same current view of what is missing. A paper list or chat thread can become stale, omit quantities, and make it hard to estimate the cost.

What the system does

Members create a family space or join one using an email-bound invitation. They can add, edit, search, filter, sort, purchase, and remove items. Each item can include a quantity, unit, category, note, and optional estimated unit price. The list shows outstanding and purchased counts and estimates the total for priced items that have not yet been purchased.

- ✓ Live updates to other open sessions
- ✓ Admin user and database tools
- ✓ Family-scoped access and invitations
- ✓ Self-hosted with Docker Compose

Core requirements

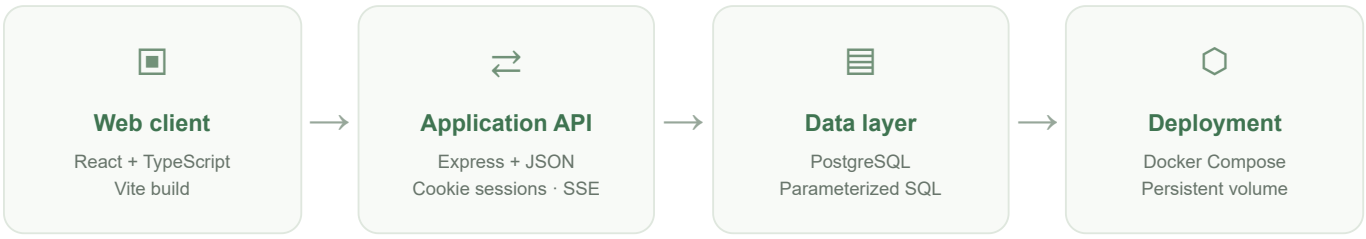
- Responsive browser UI for sign-in, family setup, and shopping.
- Validated item and account inputs, with clear feedback.
- Persistent PostgreSQL storage behind a JSON API.
- Secure sessions, role checks, and private family data.
- Near-real-time updates plus a reconnect/polling fallback.
- Repeatable build, tests, and container deployment.

React 19 · TypeScript · Vite · Express · PostgreSQL 16 · Docker

The application is designed for a single server instance. PostgreSQL is the source of truth; Server-Sent Events (SSE) notify connected browsers of family-list changes, with periodic refresh as a fallback.

02 System architecture

A deliberately small, layered design keeps the browser, HTTP API, and persistent data responsibilities separate.



Request and update flow

1. The client calls `/api` using same-origin fetch and an HTTP-only session cookie.
2. Express authenticates the user, checks family/admin scope, validates input, and queries PostgreSQL.
3. After a list change, the server emits an in-process SSE event to that family's connected clients.
4. Clients reload the authoritative list; a 30-second refresh and reconnect handling provide a fallback.

Data model

- **app_users** and **login_sessions** represent accounts and hashed session tokens.
- **families** and **family_members** scope list access and membership roles.
- **shopping_items** stores item details, purchase state, and optional prices.
- **family_invitations** stores one-time, expiring invitations as token hashes.

Security and operational choices

Passwords are bcrypt-hashed; session and invitation tokens are stored as SHA-256 hashes. Authentication uses rate limiting and strict, HTTP-only cookies. The API applies role and family checks and uses parameterized database values. Docker Compose keeps PostgreSQL on an internal network and persists its data in a named volume. For access beyond a trusted LAN, the README calls for HTTPS and secure-cookie configuration.

Runtime entry points: `src/App.tsx`, `src/lib/api.ts`, `server/index.js`, `server/schema.sql`, and `docker-compose.yml`.

03 Development process by module

The project is organized as small vertical slices: clarify the user flow, implement it at the correct boundary, then validate its behavior.

MODULE 01 Identity & family membership

Approach. Keep account identity separate from household membership. Use a single active family membership per account; let an owner invite a specific email without exposing a reusable bearer token in storage.

Workflow. Map sign-up/sign-in and first-run screens; add family create/join endpoints and membership roles; generate random invitation tokens, persist only their hashes, and enforce email, expiry, and one-time-use checks; add admin account and temporary-password flows.

Validation. Exercise login/registration form behavior and credential validation. Review authorization paths for required login, owner/admin role, invited email, and membership state. Manually verify that reset forces a password change and invalidates existing sessions.

AI tool. GitHub Copilot in VS Code was used as a code-aware assistant for repository analysis and documentation. Suggestions were checked against the existing UI, Express handlers, SQL constraints, and tests rather than accepted blindly.

Representative prompts for this documentation pass: “Trace registration from the React form to the server and database.” “Summarize invitation expiry, single-use, and email checks from the code.”

MODULE 02 Shared shopping-list interface

Approach. Build the main screen around the household's immediate task: see what is still needed, add a product quickly, and mark it purchased. Keep editing and filtering local to the client while treating the server as the source of truth.

Workflow. Define typed item and family models; implement the list, add/edit form, search/category/sort controls, purchase actions, and estimated total; show empty, loading, offline, and error states; refresh after successful mutations.

Validation. Check item constraints and price-total calculations with unit tests. Test input and purchase actions through the UI, verify purchased items do not count toward the estimate, and check layout at narrow and wide viewports.

AI tool. Copilot's repository context is useful for following React state and shared TypeScript types across files. Human review remains important for interaction behavior, accessible labels, and keeping UI expectations aligned with API validation.

Representative prompts for this documentation pass: “List the shopping screen's user actions and computed totals.” “Identify the existing tests for item validation and estimated totals.”

04 Development process by module

Backend and persistence work enforce the same rules independently of what the browser sends.

MODULE 03 API, authorization & live updates

Approach. Put shared business rules at the API boundary: authenticate each request, determine the user's family, validate every write, and constrain item operations to that family. Use SSE as a lightweight notification channel, not as a second data store.

Workflow. Add Express routes for item CRUD and account/family actions; validate and normalize payloads; use parameterized PostgreSQL queries; emit a family-specific change event after a successful write; reconnect the browser and periodically refetch to recover missed events.

Validation. Test API success, invalid input, unauthenticated and cross-family access, and SSE notification/reconnect behavior. Confirm that the UI reports server and network failures instead of presenting stale data as current.

AI tool. Copilot can accelerate route scaffolding and explain call paths. Every generated query or authorization branch still needs review against the schema, parameter binding, and family-scope checks.

Representative prompts for this documentation pass: “Explain how an item update is authorized and persisted.” “Trace the SSE event and polling fallback from server to React.”

MODULE 04 Database, deployment & verification

Approach. Let PostgreSQL enforce durable constraints and relationships; keep the app runtime simple to launch on a small private server. Make the database persistent and inaccessible from the public host port.

Workflow. Define tables, foreign keys, checks, and indexes in the startup schema; build the client assets; package the Node server and assets in Docker; connect it to PostgreSQL over a private Compose network and persist database files in a named volume.

Validation. Run the TypeScript/Vite production build and Vitest suite; start the Compose stack; check the health endpoint; verify registration, shared changes, admin controls, and restart persistence. The current repository includes focused UI and utility tests for authentication forms, Zod validation, and price totals.

AI tool. Copilot helps summarize build/test output and locate relevant config. Prefer project scripts and repeatable container checks; verify any AI explanation against actual command output.

Representative prompts for this documentation pass: “Describe how Compose isolates and persists PostgreSQL.” “Summarize what the current test files cover and what still needs an integration test.”

05 Challenges, tools & next steps

The interesting work is not just drawing a list; it is keeping several household sessions consistent without weakening data boundaries.

Biggest challenges

Consistent shared state

Multiple browsers can edit the same list. SSE provides fast notice, while a periodic reload and reconnect path help recover after dropped connections.

Correct access scope

Every item query and write must be tied to the authenticated member's family. Invitations must also be time-limited, single-use, and bound to the invited email.

Useful estimates

Optional unit prices and decimal quantities need consistent validation; purchased or unpriced products must not inflate the outstanding estimate.

Safe self-hosting

Docker networking, persistent backups, and HTTPS/proxy settings matter because the app handles account credentials and family data.

Tool reflection

Most useful: GitHub Copilot in VS Code. In this documentation pass, repository-aware assistance helped connect UI behavior, API routes, schema, and tests quickly. Suggestions remain hypotheses until checked against source and command output.

Comparison. This work did not include a controlled benchmark of Copilot against Claude Code, Augment, or Cursor. No speed or quality ranking is claimed. Copilot was the available fit because it was integrated into the current VS Code workflow.

What I would improve next

- Add API integration tests with a disposable PostgreSQL database.
- Add end-to-end coverage for invitations, simultaneous edits, and reconnect recovery.
- Improve multi-instance live updates with PostgreSQL notifications or a shared event broker.
- Add deployment monitoring and a scheduled, tested backup/restore routine.

Testing strategy and validation status

The repository's Vitest files cover sign-in/registration UI behavior, item and credential validation, and estimated-total calculations. For a release, run `npm test` and `npm run build`, then perform a Compose smoke test at `/api/health` and exercise family sharing in two browser sessions. In this capture environment, VS Code test discovery had no test controller, the shell could not find npm, and Docker's Linux engine was unavailable; therefore no fresh test/build or live-backend pass is claimed here.

06 Working-system screenshots

The screenshots show the ShopMates sign-in screen and an example family shopping list with sample household data.

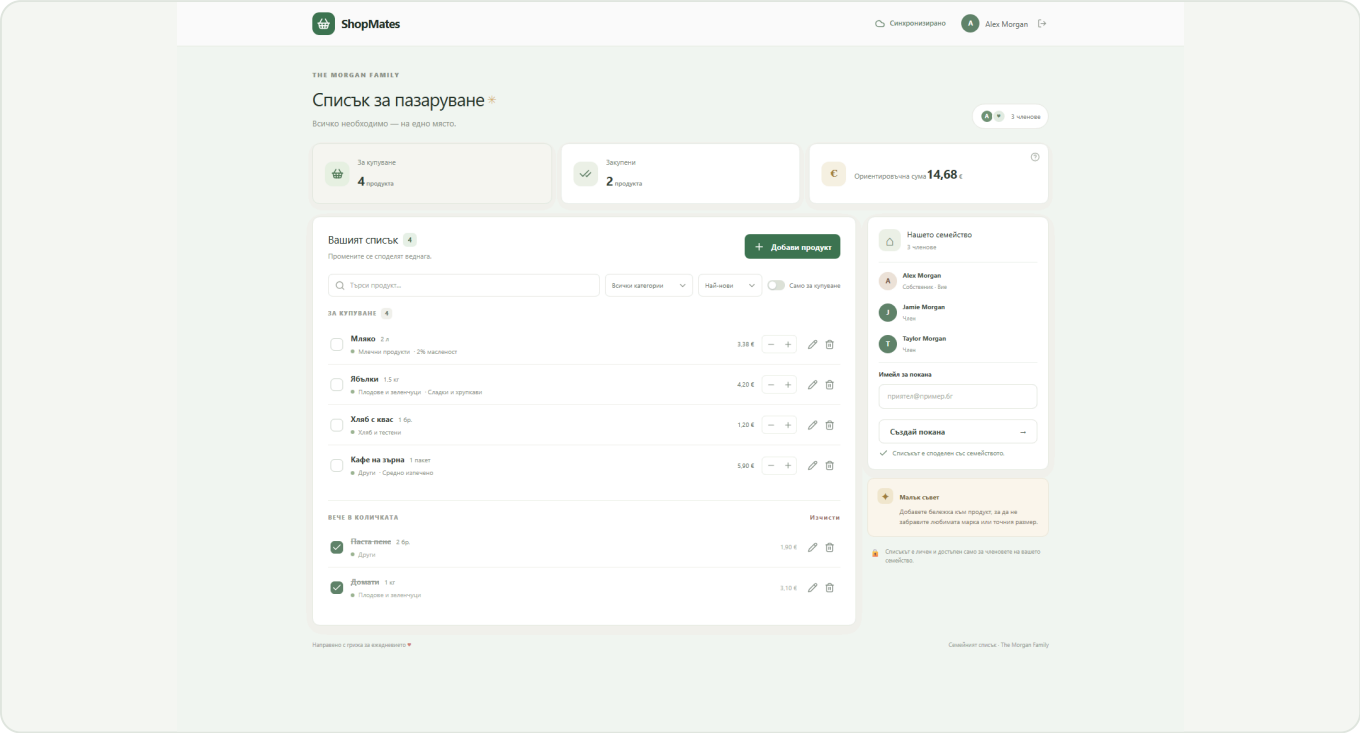


Figure 1. Example family shopping list showing sample products, quantities, estimated prices, purchase status, and household members.

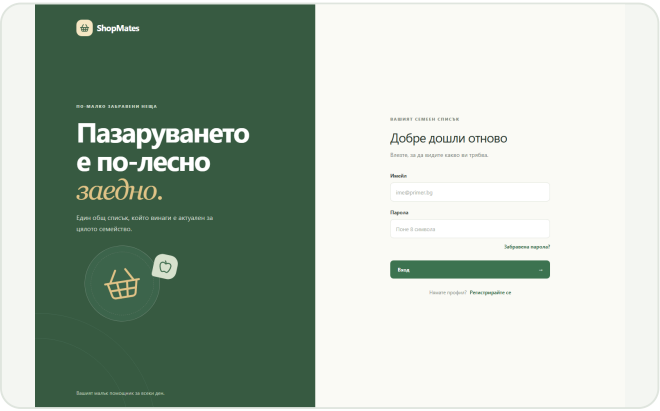


Figure 2. ShopMates sign-in screen, with registration and password-help options.

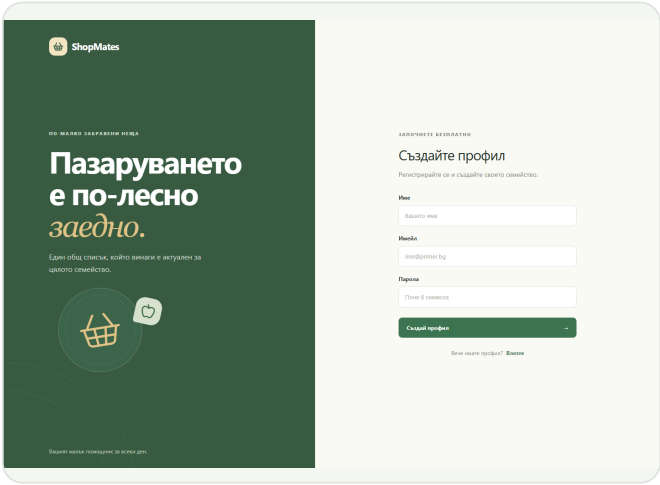


Figure 3. New users create an account with a display name, email, and password.

The examples in this report are representative prompts for the documentation pass, not a claim that every prompt was part of the original implementation history.

07 Public project repository

The source code and setup documentation for ShopMates are available at the public GitHub repository below.

GitHub repository:
https://github.com/AAssenova/ShopMates

This link opens the public source-code repository for ShopMates, a full-stack family shopping list application. It contains the React and TypeScript frontend, Express API, PostgreSQL schema, Docker configuration, and project setup instructions.